



The Art of Guard, Part 6

SELinux Policy Modules

In this article, we will learn about the basics of policy modules.

Last month, we looked at logging SELinux tasks and deciphering them using various methods—most notably *auditd* and *setroubleshoot* RPMs. We learned that we can easily fix minor errors involving Boolean values and/or changing the types of certain files by using commands like *setsebool* and *chcon/restorecon*.

But what about allowing a certain set of actions that are, by default, denied by the default targeted policy? What if we really want files of *tmp_t* to be accessed by the *httpd* daemon due to certain requirements in our work environment?

What if we wanted to install our own applications and daemons while still maintaining the security provided by SELinux? What if we run an Oracle database on our RHEL server, which is exposed to the Internet—because that is its intended use?

What about the whole concept of open source, freedom and the ability to modify applications to suit our needs rather than we modifying ourselves to suit the application's needs?

Thankfully, SELinux gives us the power and complete freedom to achieve all this. There are two ways in which we can approach the whole concept of customising SELinux:

1. Modifying the source of the policy (easily available through source RPMs).
2. Developing modules that can be compiled and loaded along with the base policy.

For beginners, intermediate-level users and also for production purposes, I would not recommend the first option unless it is absolutely necessary. It requires more in-depth knowledge and experience to modify the core

policy—we will take a look at this option in a later part of this series.

For most of us, the second option of building policy modules will suffice. It is an easier approach and also lets us develop a better understanding of the SELinux policy language before we delve deeper into it by modifying the core policy.

Customising the default policy: Power to the people

As discussed above, we will customise the default targeted policy by adding our own modules. To do so we will use the *semodule* command.

semodule is the tool used to manage SELinux policy modules, including installing, upgrading, listing and removing modules. It is installed by the *policycoreutils* RPM. The man page, as usual, gives further details and helpful instructions on how to use the command.

Listing SELinux modules

To list modules, use the following:

```
# semodule -l
```

If you execute this command on a freshly installed system with SELinux enabled, you will see a list of modules. A sample output on my system follows:

```
[root@svbg ~]# semodule -l  
amavis 1.1.0  
ccs 1.0.0  
clamav 1.1.0
```